

CS502 Midterm Current Papers Fall 2023 [Mubashir].

+ MCQs

- Few from Waqar, Moaaz & Haadi 1,2, 3 Files.

+ Short Questions

- Which one of the following is the maximal points (According to 2-D Maxima)?
- Edit-Distance between 2 Words by deleting last column.
- 3 steps of Divide & Conquer.

+ Long Questions

- Suggest how Quick Sort can be improved more.
 - Formula of Recursion
 - How many times the following given loop will execute?
-

Question: In Algorithms, write "average" and "worst" time complexity of Quick sort.

Answer: The average time complexity of QuickSort is $O(n \log n)$, while the worst-case time complexity is $O(n^2)$.

Three step or divide and conquer tha ek question

Divide: the problem into a small number of pieces

Conquer: solve each piece by applying divide and conquer to it *recursively*

Combine: the pieces together into a global solution.

Ek 6 chap ma fib Wala phla algorithm likhna tha chota c Jo ha 3 line

```
FIB(n)
1  if (n < 2)
2    then return n
3  else return FIB(n - 1) + FIB(n - 2)
```

Aur subjective abi shi yad ni lkin 2D Maxima

1.8 Example: 2-dimension maxima

Let us do an example that illustrates how we analyze algorithms. Suppose you want to buy a car. You want the pick the fastest car. But fast cars are expensive; you want the cheapest. You cannot decide which is more important: speed or price. Definitely do not want a car if there is another that is both faster and cheaper. We say that the fast, cheap car *dominates* the slow, expensive car relative to your selection criteria. So, given a collection of cars, we want to list those cars that are not dominated by any other.

Here is how we might model this as a formal problem.

- Let a point p in 2-dimensional space be given by its integer coordinates, $p = (p.x, p.y)$.
- A point p is said to be dominated by point q if $p.x \leq q.x$ and $p.y \leq q.y$.
- Given a set of n points, $P = \{p_1, p_2, \dots, p_n\}$ in 2-space a point is said to be maximal if it is not dominated by any other point in P .

Long question me 1 Fibonacci sequence ki value ko dynamic programming me krna tha

```
ITERFIB(n)
1  F[0] ← 0
2  F[1] ← 1
3  for i ← 2 to n
4  do
5    F[i] ← F[i - 1] + F[i - 2]
6  return F[n]
```

Ye 2 videos dekh lijiye ga mcqs aur usky related concepts apky clear ho jaye gaye. In Sha Allah Kafi acha attempt ho jaye ga

<https://youtu.be/CD3RCknhCQc?si=qgHD2bHq131-Mffv>

<https://youtu.be/DlmjtDs6xqQ?si=Me5Tpylu0lw323RZ>

Define edit script

In the context of algorithms, an "edit script" refers to a sequence of edit operations that transform one object into another. Edit scripts are commonly used in the field of computational biology, text processing, and computer science in general. The concept is often associated with the problem of measuring the similarity or dissimilarity between two sequences and finding the minimum set of operations (edits) needed to transform one sequence into the other.

The basic edit operations include:

1. ****Insertion (I):**** Adding a new element to the sequence.
2. ****Deletion (D):**** Removing an element from the sequence.
3. ****Substitution (S):**** Replacing one element with another.

For example, given two strings "kitten" and "sitting," an edit script to transform "kitten" into "sitting" might look like:

1. ****Insert 's' at position 0:**** "skitten"
2. ****Substitute 'i' for 'k' at position 1:**** "sitten"

3. ****Insert 'i' at position 3:**** "sitten"
4. ****Insert 't' at position 4:**** "sittiten"
5. ****Substitute 't' for 'e' at position 6:**** "sittitn"
6. ****Insert 'g' at position 8:**** "sittitng"

The minimum edit distance between two sequences is the smallest number of edit operations needed to transform one sequence into the other. Edit scripts are used to represent and analyze these transformations in the context of algorithms like the Wagner-Fisher algorithm for dynamic programming, which efficiently computes the edit distance between two sequences.

Or long ma median find karna tha

Medians are useful as a measures of the *central tendency* of a set especially when the distribution of values is highly skewed. For example, the median income in a community is a more meaningful measure than average. Suppose 7 households have monthly incomes 5000, 7000, 2000, 10000, 8000, 15000 and 16000. In sorted order, the incomes are 2000, 5000, 7000, 8000, 10000, 15000, 16000. The median income is 8000; median is element with rank 4: $(7 + 1)/2 = 4$. The average income is 9000. Suppose the income 16000 goes up to 450,000. The median is still 8000 but the average goes up to 71,000. Clearly, the average is not a good representative of the majority income levels.

How bubble sort algorithm work 3 marks

Bubble Sort is a simple sorting algorithm that repeatedly steps through the list to be sorted, compares each pair of adjacent items, and swaps them if they are in the wrong order. The pass through the list is repeated until the list is sorted. The algorithm gets its name because smaller elements "bubble" to the top of the list.

Here's a step-by-step explanation of how the Bubble Sort algorithm works:

1. ****Compare adjacent elements:**** Start at the beginning of the list and compare the first two elements. If the first element is larger than the second one, swap them.
2. ****Move to the next pair:**** Move to the next pair of elements (the second and third elements) and compare them. Again, swap them if they are in the wrong order.

3. ****Continue through the list:**** Continue comparing and swapping adjacent elements throughout the entire list. After the first pass through the list, the largest element will have "bubbled up" to the end of the list.

4. ****Repeat:**** Repeat steps 1-3 for the remaining unsorted elements. After the second pass, the second-largest element will have bubbled up to its correct position.

5. ****Continue until sorted:**** Keep repeating the process until the entire list is sorted. In each pass, the next largest element will move to its correct position.

Here's a simple example to illustrate the process:

Original list: `[5, 2, 9, 1, 5]`

1. Pass 1: `[2, 5, 1, 5, 9]` (swaps: 5 and 2, 9 and 1)

2. Pass 2: `[2, 1, 5, 5, 9]` (swaps: 5 and 1)

3. Pass 3: `[1, 2, 5, 5, 9]` (no swaps)

Now, the list is sorted.

Bubble Sort has a time complexity of $O(n^2)$, where n is the number of elements in the list. It's not the most efficient sorting algorithm, especially for large lists, but it is easy to understand and implement. There are more efficient sorting algorithms like Merge Sort or QuickSort for larger datasets.

$F=10n^2$, $F=n^3$ when these functions can be equal 3 marks

To find when the functions $F = 10n^2$ and $F = n^3$ are equal, you can set them equal to each other and solve for n :

$$10n^2 = n^3$$

Divide both sides by n^2 :

$$10 = n$$

So, the two functions are equal when $n = 10$. This means that when the input value (n) is 10, both functions $F = 10n^2$ and $F = n^3$ will have the same output.

Write the principal which differentiate between divide and conquer and dynamics programming 5 marks

1. Divide and Conquer:

- **Principle:** The "divide and conquer" paradigm involves breaking down a problem into smaller subproblems, solving the subproblems independently, and then combining their solutions to obtain the solution for the original problem.
- **Key Steps:**
 - 1.1. **Divide:** Break the problem into smaller, more manageable subproblems.
 - 1.2. **Conquer:** Solve the subproblems recursively.
 - 1.3. **Combine:** Combine the solutions of the subproblems to obtain the solution for the original problem.
- **Example:** Merge Sort and Quick Sort are classic examples of divide and conquer algorithms.

2. Dynamic Programming:

- **Principle:** The "dynamic programming" paradigm involves solving a problem by breaking it down into overlapping subproblems and solving each subproblem only once, storing the solutions to subproblems in a table to avoid redundant computations.
- **Key Steps:**
 - 2.1. **Define the Structure:** Express the problem as a recurrence relation with overlapping subproblems.
 - 2.2. **Top-Down or Bottom-Up Approach:** Solve the subproblems either recursively in a top-down fashion or iteratively in a bottom-up fashion.
 - 2.3. **Memoization or Tabulation:** Use memoization (caching results of recursive calls) or tabulation (building solutions iteratively) to avoid redundant computations.
- **Example:** The Fibonacci sequence can be efficiently computed using dynamic programming. Algorithms like Floyd-Warshall for all pairs shortest paths and Knapsack problem are also solved using dynamic programming.



1 : Problem selection k steps likhny thy

1. **Input:**

- Receive the input parameters, including the set of elements (array or list) and the value of k (the position of the desired element).

2. **Partitioning:**

- Choose a pivot element from the set. Partition the set into two sub-arrays: elements smaller than the pivot and elements greater than the pivot.

3. **Recursion:**

- Determine which partition contains the k th element based on the current position of the pivot.
- If k is less than the size of the smaller partition, recursively apply the selection algorithm on the smaller partition.
- If k is equal to the size of the smaller partition, return the pivot as the k th element.
- If k is greater than the size of the smaller partition, recursively apply the selection algorithm on the larger partition with adjusted k .

4. **Base Case:**

- If the size of the set becomes small enough (e.g., a constant size), solve the problem using a simple algorithm like sorting the small set and returning the k th element.

5. **Iteration:**

- Repeat the process iteratively until the k th element is found.

3 : Ak table tha us me tmam values ki asymptotic notation likhni thi.

Here is a list of common asymptotic running times:

- $\Theta(1)$: Constant time; can't beat it!
- $\Theta(\log n)$: Inserting into a balanced binary tree; time to find an item in a sorted array of length n using binary search.
- $\Theta(n)$: About the fastest that an algorithm can run.
- $\Theta(n \log n)$: Best sorting algorithms.
- $\Theta(n^2)$, $\Theta(n^3)$: Polynomial time. These running times are acceptable when the exponent of n is small or n is not too large, e.g., $n \leq 1000$.
- $\Theta(2^n)$, $\Theta(3^n)$: Exponential time. Acceptable only if n is small, e.g., $n \leq 50$.
- $\Theta(n!)$, $\Theta(n^n)$: Acceptable only for really small n , e.g. $n \leq 20$.

4: memoization k bary me 5 marks ka question tha

We can avoid this unnecessary repetitions by writing down the results of recursive calls and looking them up again if we need them later. This process is called *memoization*. Here is the algorithm with memoization.

```
MEMOFIB(n)
1  if (n < 2)
2    then return n
3  if (F[n] is undefined)
4    then F[n] ← MEMOFIB(n - 1) + MEMOFIB(n - 2)
5  return F[n]
```

Aik question tha how we can Build heap

4.2.5 BuildHeap

We can use Heapify to build a heap as follows. First we start with a heap in which the elements are not in heap order. They are just in the same order that they were given to us in the array A . We build the heap by starting at the leaf level and then invoke Heapify on each node. (Note: We cannot start at the top of the tree. Why not? Because the precondition which Heapify assumes is that the entire tree rooted at node i is already in heap order, except for i .) Actually, we can be a bit more efficient. Since we know that each leaf is already in heap order, we may as well skip the leaves and start with the first non-leaf node. This will be in position $n/2$. (Can you see why?)

Here is the code. Since we will work with the entire array, the parameter m for Heapify, which indicates the current heap size will be equal to n , the size of array A , in all the calls.

```
BUILDHEAP(array A, int n)
1  for i ← n/2 downto 1
2    do
3      HEAPIFY(A, i, n)
```

1#Seive technique ko selection problem and median sy related working btani thi iski

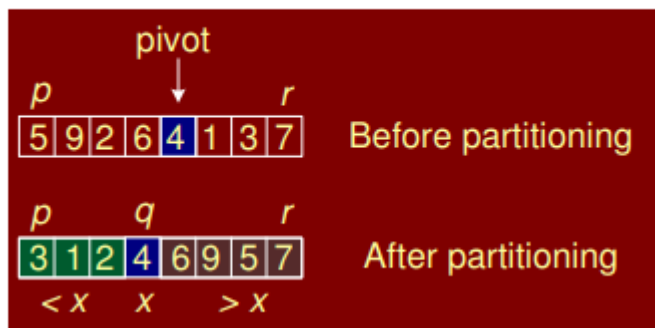
3.2.2 Applying the Sieve to Selection

To see more concretely how the sieve technique works, let us apply it to the selection problem. We will begin with the given array $A[1..n]$. We will pick an item from the array, called the *pivot element* which we will denote by x . We will talk about how an item is chosen as the pivot later; for now just think of it as a random element of A .

We then *partition* A into three parts.

1. $A[q]$ contains the pivot element x ,
2. $A[1..q-1]$ will contain all the elements that are less than x and
3. $A[q+1..n]$ will contain all elements that are greater than x .

Within each sub array, the items may appear in any order. The following figure shows a partitioned array:



2#Can we sort array without using combination based algorithms if yess then how?

Yes, it is indeed possible to sort an array without using combination-based algorithms.

The most common comparison-based sorting algorithms include bubble sort, insertion sort, selection sort, merge sort, and quicksort. These algorithms rely on comparisons between elements to determine their relative order.

3#how many ways used to measure time complexity??

1. **Big O Notation (O):** This is the most commonly used notation. It describes the upper bound of an algorithm's running time. In other words, it provides an asymptotic upper bound on the growth rate of the algorithm's time complexity. For example, if an algorithm has a time complexity of $O(n)$, it means that the running time grows linearly with the size of the input.
Example: $O(1)$ - constant time, $O(\log n)$ - logarithmic time, $O(n)$ - linear time, $O(n^2)$ - quadratic time, etc.
2. **Omega Notation (Ω):** This notation represents the lower bound of an algorithm's running time. It provides an asymptotic lower bound on the growth rate of the time complexity. For example, if an algorithm has a time complexity of $\Omega(n)$, it means that the running time grows at least linearly with the size of the input.
Example: $\Omega(1)$ - constant lower bound, $\Omega(n)$ - linear lower bound, $\Omega(n \log n)$ - linearithmic lower bound, etc.
3. **Theta Notation (Θ):** This notation provides both upper and lower bounds on an algorithm's running time. It bounds the growth rate of the running time both from above and below. If an algorithm has a time complexity of $\Theta(n)$, it means that the running time grows exactly linearly with the size of the input.
Example: $\Theta(1)$ - constant time, $\Theta(n)$ - linear time, $\Theta(n \log n)$ - linearithmic time, etc.

1#Brute force and plan sweep comparison?

1. Algorithmic Strategy:

- **Brute-Force:** Brute-force algorithms typically involve checking all possible pairs or combinations of geometric objects to find a solution. It systematically examines every candidate solution.
- **Plane-sweep:** Plane-sweep algorithms, on the other hand, use a sweeping line or plane to process the geometric objects in a specific order. The algorithm maintains a state as the line sweeps through the objects.

2. Efficiency:

- **Brute-Force:** Brute-force algorithms have a straightforward implementation and consider all possible combinations, but they can be inefficient for large datasets, leading to a higher time complexity.
- **Plane-sweep:** Plane-sweep algorithms are often more efficient, especially for problems where the number of interactions or events is significantly reduced by using the sweeping line or plane.

3. Data Structures:

- **Brute-Force:** Brute-force algorithms usually rely on basic data structures and may involve nested loops for comparing all pairs of objects.
- **Plane-sweep:** Plane-sweep algorithms often require more sophisticated data structures, such as sweep-line status structures or event queues, to efficiently process and manage the geometric objects as the sweeping line progresses.

4. Event Handling:

- **Brute-Force:** In brute-force algorithms, there is no concept of events or event handling. Each comparison is made explicitly without considering the geometric objects' order.
- **Plane-sweep:** Plane-sweep algorithms are event-driven. Events occur as the sweeping line encounters specific points or events in the geometric space, and the algorithm reacts to these events to update the state and find the solution.

5. Applications:

- **Brute-Force:** Brute-force algorithms are often used for small-scale problems or as a baseline approach for validating other algorithms. They are suitable when the problem size is small and the computational cost is acceptable.
- **Plane-sweep:** Plane-sweep algorithms are particularly well-suited for problems that involve intersecting geometric objects, such as line segment intersection, closest pair of points, or detecting overlapping intervals. The sweeping line helps reduce the number of pairwise comparisons and improves the overall efficiency for these types of problems.

2#3 names of Slow sorting algo and their worst case running time?

4.1 Slow Sorting Algorithms

There are a number of well-known slow $O(n^2)$ sorting algorithms. These include the following:

Bubble sort: Scan the array. Whenever two consecutive items are found that are out of order, swap them. Repeat until all consecutive items are in order.

Insertion sort: Assume that $A[1..i - 1]$ have already been sorted. Insert $A[i]$ into its proper position in this sub array. Create this position by shifting all larger elements to the right.

Selection sort: Assume that $A[1..i - 1]$ contain the $i - 1$ smallest elements in sorted order. Find the smallest element in $A[i..n]$ Swap it with $A[i]$.

These algorithms are easy to implement. But they run in $\Theta(n^2)$ time in the worst case.

3#define recursive Fibonacci numbers

In computer science and mathematics, the Fibonacci sequence is a series of numbers in which each number (Fibonacci number) is the sum of the two preceding ones, usually starting with 0 and 1. The sequence goes: 0, 1, 1, 2, 3, 5, 8, 13, 21, and so on.

2.write average and worst time complexity of quick sort

Average-case:

$\Theta(n \log n)$.

Worst Case:

$\Theta(n^2)$

2why plane sweep is better than brute-force

Plane sweep algorithms are often more efficient than brute-force methods in solving geometric problems because they exploit the inherent structure of the problem. Instead of exhaustively checking all possible pairs of elements, plane sweep algorithms simulate the movement of a sweeping line or plane across the input, processing events as they occur. This approach reduces the overall number of comparisons and computations, taking advantage of spatial and temporal relationships in the data. By efficiently handling events in a systematic order, plane sweep algorithms can achieve better time complexity and scalability compared to brute-force approaches, making them particularly well-suited for geometric problems involving points, lines, or other spatial entities.

1) algorithm or us ky advantages

An algorithm is a step-by-step procedure or a set of well-defined rules for solving a specific problem or accomplishing a particular task. It serves as a blueprint for performing computations and solving problems in a systematic way. Algorithms are fundamental to computer science and are used in various applications, ranging from sorting and searching data to complex tasks like artificial intelligence and cryptography.

Here are five advantages of algorithms:

1. **Efficiency:** Algorithms are designed to solve problems in an efficient manner, ensuring that the solution is achieved with the least amount of resources (time, memory, and computational power). Efficient algorithms contribute to faster execution and improved performance.
2. **Reusability:** Once developed, algorithms can be reused in different contexts or applications, providing a modular and scalable approach to problem-solving. This reusability not only saves time but also promotes consistency and reliability across different software systems.

3. **Clarity and Understandability:** Well-designed algorithms are often written in a clear and understandable manner. This clarity facilitates communication among developers, making it easier for them to understand, modify, or improve the algorithm. Clear algorithms also aid in documentation and maintenance.
4. **Optimization:** Algorithms can be optimized to enhance their performance based on the specific characteristics of the problem they address. Optimization may involve refining the algorithmic steps, choosing more efficient data structures, or leveraging parallel processing. This adaptability allows for tailoring algorithms to suit different requirements.
5. **Accuracy and Precision:** Algorithms are designed to produce accurate and precise results when applied to a given problem. Through rigorous analysis and testing, developers can ensure that an algorithm reliably provides the correct output under various conditions. This predictability is crucial, especially in applications where correctness is critical, such as in financial systems or medical software.

Edit distance applications k 2 names

6.3.1 Edit Distance: Applications

There are numerous applications of the Edit Distance algorithm. Here are some examples:

Spelling Correction

If a text contains a word that is not in the dictionary, a 'close' word, i.e. one with a small edit distance, may be suggested as a correction. Most word processing applications, such as Microsoft Word, have spelling checking and correction facility. When Word, for example, finds an incorrectly spelled word, it makes suggestions of possible replacements.

Plagiarism Detection

If someone copies, say, a C program and makes a few changes here and there, for example, change variable names, add a comment or two, the edit distance between the source and copy may be small. The edit distance provides an indication of similarity that might be too close in some situations.

Computational Molecular Biology DNA is a polymer. The monomer units of DNA are nucleotides, and the polymer is known as a "polynucleotide." Each nucleotide consists of a 5-carbon sugar (deoxyribose), a nitrogen containing base attached to the sugar, and a phosphate group.

Dynamic programming

6.2 Dynamic Programming

Dynamic programming is essentially recursion without repetition. Developing a dynamic programming algorithm generally involves two separate steps:

- **Formulate problem recursively.** Write down a formula for the whole problem as a simple combination of answers to smaller subproblems.
- **Build solution to recurrence from bottom up.** Write an algorithm that starts with base cases and works its way up to the final solution.